UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE - UFRN
CENTRO DE TECNOLOGIA
DEPARTAMENTO DE ENGENHARIA DE COMPUTAÇÃO E AUTOMAÇÃO
CURSO DE ENGENHARIA DE COMPUTAÇÃO

GILVANDRO CÉSAR SANTOS DE MEDEIROS

BUSINESS STRATEGY COPILOT: UMA PROVA DE CONCEITO DE ASSISTENTE INTELIGENTE USANDO MODELO SMALL LANGUAGE MODEL (SLM) PARA O PLANEJAMENTO ESTRATÉGICO DE STARTUPS

NATAL-RN
2025

GILVANDRO CÉSAR SANTOS DE MEDEIROS

BUSINESS STRATEGY COPILOT: UMA PROVA DE CONCEITO DE ASSISTENTE INTELIGENTE USANDO MODELO SMALL LANGUAGE MODEL (SLM) PARA O PLANEJAMENTO ESTRATÉGICO DE STARTUPS

Trabalho de Conclusão de Curso apresentado ao Departamento de Engenharia de Computação e Automação do Centro de Tecnologia da Universidade Federal do Rio Grande do Norte como parte dos requisitos para a obtenção do título de Bacharel em Engenharia de Computação.

Orientador: Prof. Dr. Orivaldo Vieira de Santana Júnior

Natal-RN
2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Medeiros, Gilvandro César Santos de.
Business strategy copilot: uma prova de conceito de assistente inteligente usando modelo Small Language Model (SLM) para o planejamento estratégico de startups / Gilvandro César Santos de Medeiros. - Natal, 2025.
51f.: il.

Monografia (Graduaçção) - Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Engenharia de Computação.
Orientação: Dr. Orivaldo Vieira de Santana Junior.

1. Large Language Model (LLM) - Monografia. 2. Small Language Model (SLM) - Monografia. 3. IA Generativa - Monografia. 4. Processamento de Linguagem Natural - Monografia. 5. Inovação - Monografia. I. Santana Junior, Orivaldo Vieira de. II. Título.

RN/UF/BCZM CDU 004

Elaborado por Raimundo Muniz de Oliveira - CRB-15/429

BUSINESS STRATEGY COPILOT: UMA PROVA DE CONCEITO DE ASSISTENTE INTELIGENTE USANDO MODELO SMALL LANGUAGE MODEL (SLM) PARA O PLANEJAMENTO ESTRATÉGICO DE STARTUPS

GILVANDRO CÉSAR SANTOS DE MEDEIROS

Trabalho de conclusão de curso aprovado em 20 de janeiro de 2025, pela banca examinadora composta pelos seguintes membros:

Prof. Dr. Orivaldo Vieira de Santa Júnior – Orientador

UFRN

Prof. Dr. Rodrigo César Santiago – Examinador interno

UFRN

Me. e Eng. Mateus Arnaud Santos de Sousa Goldbarg – Examinador externo

Pesquisador Especialista em Projeto de PD&I UFRN - KAROON - GEOWELLEX

Natal-RN

2025

AGRADECIMENTOS

Agradeço primeiramente à Deus, por ter me dado todas as forças, a saúde, as oportunidades e o discernimento que precisei para trilhar este caminho. Mais ainda, por ter me feito encontrar pessoas tão maravilhosas e inspiradoras nesta trajetória, todas elas importantes na construção da pessoa e do profissional que me tornei.

Aos meus pais, Gilvan Medeiros e Adenira Chianca, por terem sido meus maiores exemplos de respeito, honestidade e de esforço, que tanto me inspiraram em toda minha vida.

À minha irmã, Amanda Liz, que sempre acreditou em mim. Sua trajetória de crescimento também não será fácil, mas te darei todo o suporte para que você também possa construir o seu futuro. Que nunca te falte planejamento estratégico para te guiar em busca de seus objetivos.

À minha esposa, Bárbara Sayonnara, por ter estado ao meu lado e me apoiado mesmo nos momentos mais estressantes que tive nos últimos anos. É em você que encontrei meu porto seguro, o complemento dos meus objetivos e a vontade de ir sempre além. Desejo a nós todo o amor, saúde e felicidade que merecemos, por toda a vida.

Ao meu orientador, Professor Dr. Orivaldo Vieira de Santana Júnior, que me estendeu a mão anos antes de termos qualquer vínculo formal, me abrindo as portas que eu precisava para descobrir o meu caminho. Foi e é um dos grandes responsáveis pela minha formação acadêmica, humana e profissional, me incentivando e me apoiando em todos os momentos que precisei, a quem devo eterna gratidão.

Agradeço, de forma geral, a todos que contribuíram na minha formação, aqueles com que pude conversar e aprender, aos amigos, colegas e parceiros, que fizeram e fazem parte da minha história, e, portanto, fazem parte também deste trabalho.

RESUMO

Large Language Models (LLMs) são modelos de processamento de linguagem natural modernos, que estão por trás de tecnologias como Gemini (Google), Copilot (Microsoft) e ChatGPT (OpenAI). No entanto, LLMs comumente podem ter uma alta exigência computacional e energética, de modo a tornar o desenvolvimento de soluções baseadas nestas tecnologias algo dependente de grandes companhias de tecnologia. Neste contexto, o objetivo principal deste trabalho é explorar *Small Language Models* (SLMs), especificamente o modelo pré-treinado Phi3-Mini, como uma alternativa relevante e eficiente, capaz de fornecer respostas inteligentes em problemas complexos e com menor demanda de recursos. Assim, em um ambiente de programação Python com o suporte da plataforma *open-source* Ollama, configurado em um laptop Dell Inspiron 15 5510, foi desenvolvida uma aplicação para auxiliar o planejamento estratégico de startups. A aplicação Python combina boas práticas de orientação a objetos e *prompt engineering*, permitindo gerar, refinar e priorizar objetivos estratégicos com base em dados como missão, visão e clientes da *startup*, aplicando controle de qualidade das respostas. Um estudo de caso evidenciou a eficiência do modelo em hardware local, e todo o código-fonte foi disponibilizado publicamente no Github, fomentando o uso de IA de menor custo computacional e energético para fins de gestão, empreendedorismo e inovação.

Palavras-chave: Large Language Model (LLM). Small Language Model (SLM). IA Generativa. Processamento de Linguagem Natural. Inovação.

ABSTRACT

Large Language Models (LLMs) are modern natural language processing models that are behind technologies such as Gemini (Google), Copilot (Microsoft), and ChatGPT (OpenAI). However, LLMs can often have high computational and energy requirements, making the development of solutions based on these technologies dependent on large technology companies. In this context, the main objective of this work is to explore Small Language Models (SLMs), specifically the pre-trained Phi3-Mini model, as a relevant and efficient alternative, capable of providing intelligent responses to complex problems and with lower resource demands. Thus, in a Python programming environment supported by the open-source Ollama platform, configured on a Dell Inspiron 155510 laptop, an application was developed to assist the strategic planning of startups. The Python application combines good practices in object-oriented programming and prompt engineering, allowing the generation, refinement, and prioritization of strategic objectives based on data such as the startup's mission, vision, and customers, applying quality control to the responses. A case study demonstrated the efficiency of the model on local hardware, and all source code was made publicly available on Github, encouraging the use of AI with lower computational and energy costs for management, entrepreneurship and innovation purposes.

Keywords: Large Language Model (LLM). Small Language Model (SLM). Generative AI. Natural Language Processing. Innovation.

LISTA DE ILUSTRAÇÕES

FIGURA		PÁGINA
1	O Método BSE em Etapas	19
2	Inicialização da classe *StartupStrategicPlanning* no estudo de caso realizado	33
3	Rotina completa de utilização do modelo e dos métodos desenvolvidos	34

LISTA DE TABELAS

TABELA		PÁGINA
1	Configurações de dispositivo utilizado no trabalho	20
2	Definições adotadas para melhor direcionar o modelo	25
3	Formato do prompt para o método response_quality_control	28
4	Formato do prompt para o método review_specific_response	29
5	Formato do prompt para o método review_specific_response	30
6	Formato do prompt para o método review_specific_response	31
7	Formato do prompt para o método refine_strategic_objectives	31
8	Formato do prompt para o método priority_objectives	32
9	Informações básicas da startup após chamada do método fill_informations	34
10	Informações da startup revisadas pelo método response_quality_control - Etapa 1	35
11	Informações da startup revisadas pelo método response_quality_control - Etapa 2	36
12	Modificações realizadas pelo modelo no campo de “Visão”	39
13	Objetivos estratégicos obtidos na primeira iteração	40
14	Objetivos estratégicos refinados	41
15	Objetivos estratégicos ordenados em ordem de prioridade	42
16	Diferenças no tempo de processamento para cada execução do modelo	43
17	Objetivos estratégicos ordenados em ordem de prioridade [2000 tokens]	44
18	Objetivos estratégicos ordenados em ordem de prioridade [20000 tokens]	45

LISTA DE ABREVIATURAS E SIGLAS

SIGLA	SIGNIFICADO
API	*Application Programming Interface*
BSE	*Business Strategy Engineering*
GPT	*Generative Pre-trained Transformer*
IA	Inteligência Artificial
LLM	*Large Language Model*
LLaVA	*Large Language and Vision Assistant*
LTS	*Long-Term Support* (Suporte de Longo Prazo)
NLP	*Natural Language Processing* (Processamento de Linguagem Natural)
OPT	*Open Pre-trained Transformer*
PD&I	Pesquisa, Desenvolvimento e Inovação
POO	Programação Orientada a Objetos
SLM	*Small Language Model*

SUMÁRIO

1 INTRODUÇÃO **11**
1.1 Objetivos 13
2 FUNDAMENTAÇÃO TEÓRICA **14**
2.1 Large Language Models (LLMs) 14
2.2 Small Language Models (SLMs) 15
2.3 Modelo Phi3-Mini e Framework Ollama 16
2.4 Prompt Engineering 16
2.5 Linguagem de Programação Python e Programação Orientada a Objetos 17
2.6 Noções Gerais: Startups, Planejamento Estratégico e Abordagem Business Strategy Engineering (BSE) 18
3 – METODOLOGIA **20**
3.1 Configuração de Ambiente 20
3.2 Desenvolvimento de Código 21
4 – PROVA DE CONCEITO, RESULTADOS E DISCUSSÕES **33**
4.1 Estudo de Caso Completo - Modelo em Mínima Performance 33
4.2 Estudo de Caso Comparativo - Mudanças no Parâmetro Performance 43
5 – CONSIDERAÇÕES FINAIS **47**
REFERÊNCIAS **49**

1 INTRODUÇÃO

A área de processamento de linguagem natural (*Natural Language Processing* – NLP), como um ramo da Inteligência Artificial (IA), vem ganhando espaço rapidamente no mercado nos últimos anos, sobretudo impulsionada pelo surgimento dos *Large Language Models* (LLMs), modelos de NLP capazes de generalizar múltiplas funções e gerar textos com comunicação coerente (Rillig *et al.*, 2023). Fazem parte desta ascensão tecnológica assistentes digitais modernos, como o ChatGPT - OpenAI, Gemini - Google e Copilot - Microsoft (Naveed *et al.*, 2023).

Os LLMs são modelos com arquiteturas complexas e um grande número de parâmetros ajustáveis, treinados em volumes massivos de informações, adquirindo assim a capacidade de gerar textos coesos a partir de diferentes entradas fornecidas por um usuário (Rillig *et al.*, 2023). Este processo de interação com o modelo e personalização de resultados obtidos é realizado através de uma interação textual, mediante o envio de *prompts*, orientações usualmente textuais para o modelo, que são interpretadas e convertidas em respostas. Este processo iterativo através do envio de *prompts*, muitas vezes expressos em linguagem natural, torna os LLMs uma tecnologia capaz de interagir com seres humanos em um processo conversacional, o que facilita a sua adoção e aceitação no mercado para variados propósitos (Rillig *et al.*, 2023).

Assim, LLMs apresentam habilidades emergentes e úteis, com alguns dos modelos mais avançados conhecidos chegando a executar ações que envolvem raciocínio, planejamento, tomada de decisões e um aprendizado contínuo, através de ajustes para contextos específicos, mesmo no caso em que o modelo não foi especificamente treinado para aquele determinado contexto (Naveed *et al.*, 2023).

No entanto, as capacidades dos LLMs de resolver tarefas diversas com desempenho próximo ao humano em diferentes áreas têm um custo: tratam-se de processos de treinamento exaustivos e custosos, processos de inferência lentos, elevados requisitos de *hardware*, resultando em elevados custos operacionais, custos energéticos e, por consequência, riscos ambientais (Naveed *et al.*, 2023; Rillig *et al.*, 2023).

Estas limitações e consequências decorrentes dos LLMs têm impulsionado novas linhas de pesquisa e aplicações buscando o desenvolvimento de arquiteturas mais eficientes e estratégias de treinamento aprimoradas (Naveed *et al.*, 2023). Nesse contexto, têm surgido

modelos alternativos chamados de *Small Language Models* (SLMs), modelos baseados em LLMs, mas que buscam otimizar processos e encontrar equilíbrio entre qualidade de respostas e demanda por recursos (Zhu *et al.*, 2024). Tais modelos permitem a execução de SLMs em *hardwares* mais acessíveis, como laptops e computadores de uso pessoal, mantendo boa parte de suas capacidades sem depender de grandes infraestruturas em nuvem.

Nesse contexto, o presente trabalho realiza um estudo de caso com o modelo *open source* de LLM pré-treinado Phi3-Mini, disponibilizado ao mercado pela Microsoft, e acessível através da plataforma Ollama, também de código aberto, que permite uma maior facilidade na instalação e utilização de modelos de LLM, mesmo em dispositivos pessoais (Microsoft Latam, 2025).

Nesse estudo de caso, foi selecionado um problema-alvo específico e desafiador, dentro do contexto do planejamento estratégico de startups, baseado na abordagem *Business Strategy Engineering*, discutida em Medeiros (2023). Esta metodologia combina fundamentos científicos e diferentes métodos para o planejamento estratégico, estruturando em um processo por etapas, que permite a empresas com foco em inovação / *startups* obterem objetivos estratégicos relevantes, considerando para isso variáveis internas e externas. Este problema foi selecionado pois, para ter capacidade de lidar com estas informações, o modelo Phi3-Mini deve ter também capacidade de lidar com um problema amplo, de múltiplas variáveis e perspectivas, de modo a avaliar assim a sua capacidade de interpretação e generalização para um problema significativamente fora do escopo para o qual este modelo foi treinado.

Para isso, foi desenvolvida uma aplicação Python *backend*, realizando a interação com o modelo Phi3-Mini em um processo estruturado de envio de *prompts* e controle de qualidade nas respostas informadas, executado através de diferentes métodos e controle de variáveis, aplicando conceitos da programação orientada a objetos. Foi utilizado apenas recursos computacionais locais, oriundos de um dispositivo básico para os padrões de mercado atuais, sendo um laptop Dell Inspiron 15 5510, com processador Intel Core i7 - 11th Gen, 16 GB de RAM, placa de vídeo integrada Intel Iris Xe e sistema operacional Ubuntu 24.04.1 LTS.

Os resultados obtidos por este trabalho sugerem que o modelo Phi3-Mini performa de maneira relativamente eficiente e rápida, e tem uma capacidade de generalização relevante, quando assistido por *prompts* resumidos, bem formatados e, sobretudo, quando assistido por uma estrutura lógica que divida o problema por etapas, uma abordagem que envolve técnicas de desenvolvimento convencionais com modelos generativos avançados, facilitando a tomada de decisão.

1.1 Objetivos

O presente trabalho tem como objetivo geral explorar a utilização do modelo SLM Phi3-Mini, como uma alternativa eficiente e acessível para lidar com problemas complexos, especificamente voltados para uma prova de conceito na aplicação de um método de planejamento estratégico para *startups*, avaliando assim sua capacidade de interpretar e gerar soluções relevantes com baixa exigência computacional.

Dentro do escopo definido por este objetivo geral, os seguintes objetivos específicos foram definidos:

- Desenvolver uma aplicação Python orientada a objetos que permite interagir com o modelo Phi3-Mini de forma estruturada, incorporando boas práticas de engenharia de software e técnicas de *prompt engineering* para maximizar a qualidade das respostas geradas.
- Implementar um fluxo lógico que permita partir de informações gerais sobre a startup, incluindo missão, visão, fase de maturidade e clientes, fazendo com que o modelo selecionado interprete tais informações, auxilie no seu preenchimento e delimitação, e possa partir destas informações para sugerir objetivos estratégicos, visando o sucesso daquela empresa analisada.
- Avaliar, na prova de conceito realizada, a capacidade do modelo Phi3-Mini de interpretar e processar múltiplas variáveis relacionadas ao planejamento estratégico, identificando a qualidade das respostas em cenários complexos que demandem raciocínio, interpretação e generalização.
- Avaliar o consumo de tempo da aplicação desenvolvida, variando parâmetros de inicialização do modelo Phi3-Mini, e investigando o impacto destas variações no tempo de resposta dado pelo modelo.
- Publicar o código-fonte da aplicação desenvolvida em um repositório público no GitHub, fornecendo uma base de código que poderá ser reutilizada por outros pesquisadores e empreendedores interessados em inovação e inteligência artificial.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo aborda as bases de conhecimento e referenciais teóricos que fundamentam o projeto aqui relatado, seu processo de desenvolvimento e a interpretação dos resultados apresentados, assim como uma melhor compreensão das ferramentas e tecnologias utilizadas.

2.1 Large Language Models (LLMs)

Large Language Models (LLMs) são uma classe de modelos de inteligência artificial recentes e em ascensão, que vêm demonstrando capacidade de processar e gerar textos com comunicação coerente e generalizar para múltiplas tarefas (Naveed *et al.*, 2023). O fundamento por trás destes modelos vêm da evolução da arquitetura *Transformer*, um tipo de modelo de aprendizado profundo (*deep learning*) altamente paralelizável, com capacidade de representar blocos de informações, como palavras, em *tokens*, unidades básicas de texto, que podem ser então processadas e analisadas em contexto,. Desta maneira, com a análise sequencial de *tokens* que representam as informações textuais, podem ser atribuídos significados para este sequenciamento, que podem ser aprendidos pelo modelo, de modo que uma vez exposto a sequências similares no futuro, o modelo possa replicar este comportamento aprendido (Zhao *et al.*, 2023).

Desta maneira, começou-se a investir esforços direcionados para o desenvolvimento de modelos *Transformers* pré-treinados, aumentando cada vez mais o número de parâmetros ajustáveis nestes modelos, o volume de dados para os quais eles eram expostos e o número de *tokens* processados, de modo, assim, a observar modelos com capacidade adaptativa cada vez mais ampla para processamento de problemas relacionados ao processamento de linguagem natural em áreas diversas (Zhao *et al.*, 2023). Assim, alcançou-se um novo patamar para os modelos de NLP, definindo a classe de modelos pré-treinados denominada por LLMs (Naveed *et al.*, 2023). LLMs são, atualmente, treinados de forma autossupervisionada, sendo expostos a um volume significativo de textos, com o objetivo de aprender representações genéricas que podem ser compartilhadas para variados contextos (Naveed *et al.*, 2023).

O potencial e as capacidades destes modelos vêm gerando uma série de resultados promissores, mas, ao mesmo tempo, vêm apresentando desafios e problemáticas quanto ao seu uso (Rillig *et al.*, 2023). Um dos pontos a se considerar é o excessivo consumo energético e, por consequência, a intensa emissão de carbono para a atmosfera, tanto no processo de

treinamento quanto na utilização em massa destes modelos, que estão com um número cada vez maior de parâmetros - tornando seu custo computacional e, no fim, energético cada vez mais intenso (Rillig *et al.*, 2023).

Segundo Rilling et al. (2023), a pegada de carbono decorrente da ascensão dos modelos de LLM dependerá do consumo energético e da origem da fonte de energia utilizada. Além das emissões de dióxido de carbono, os grandes servidores e *data centers* podem gerar, também, outros impactos ambientais, como a contaminação de água utilizada no resfriamento destas grandes unidades de processamento e o impacto ambiental decorrente da construção de instalações computacionais de cada vez maior porte. Assim, surge a necessidade de investir esforços em modelos com capacidades similares às LLMs, mas com menor consumo energético e menor impacto ambiental.

2.2 Small Language Models (SLMs)

Embora os avanços recentes tenham privilegiado modelos cada vez maiores de LLMs, o potencial de treinar modelos menores com conjuntos de dados maiores permanece subexplorado (Zhang *et al.*, 2024). Touvron et al. (2023) demonstra que modelos menores, quando treinados com mais dados, podem se aproximar, igualar ou até superar seus equivalentes maiores.

Um dos modelos conhecidos, alinhados com esta linha de desenvolvimento de SLMs, é o TinyLlama, discutido e apresentado por Zhang *et al.* (2024). Neste trabalho, os autores propuseram o modelo TinyLlama, como um modelo compacto de LLM (de cerca de 1 bilhão de parâmetros ajustáveis), que utilizando a estratégia de maximizar o número de *tokens* (chegando a até 3 trilhões de *tokens*) pode performar tão bem - ou até melhor - que outros modelos com maior número de parâmetros, como o OPT-1.3B (Zhang *et al.*, 2022) e o Pythia-1.4B (Biderman *et al.*, 2023).

Zhu *et al.* (2024) também mostra uma abordagem interessante para SLMs, apresentando o modelo LLaVA-ϕ (LLaVA-Phi), um assistente multimodal que utiliza o modelo SLM Phi-2 em uma abordagem multimodal, considerando diálogos complexos que integram elementos textuais e visuais (imagens). Zhu *et al.* (2024) mostra que modelos pequenos, com apenas 2,7 bilhões de parâmetros, podem performar bem em diálogos e desafios complexos, comparável a modelos de 3 vezes o número de parâmetros, desde que sejam treinados com conjuntos de dados de alta qualidade. Comparativamente, o já

ultrapassado modelo GPT-3, da OpenAI, foi treinado com cerca de 175 bilhões de parâmetros (Naveed *et al.*, 2023).

2.3 Modelo Phi3-Mini e Framework Ollama

Ainda no contexto de SLM, em abril de 2024 a Microsoft anunciou uma nova família de modelos abertos pré-treinados de LLM, a família Phi-3, composta pelos modelos Phi3-medium, com 14 bilhões de parâmetros, Phi3-small, com 7 bilhões de parâmetros, e o Phi3-Mini, com 3,8 bilhões de parâmetros (MICROSOFT LATAM, 2025).

O trabalho de Abdin *et al.* (2024) apresenta em mais detalhes o modelo phi3-mini, rivalizando com o modelo GPT-3.5, de 175 bilhões de parâmetros, em *benchmarkings* realizados. Os autores comentam, ainda, dos esforços realizados para aprimorar a qualidade da base de dados utilizada para treinar o modelo Phi-3, alegando que a filtragem de dados públicos e a criação de dados sintéticos permite que modelos menores apresentem desempenho comparável a modelos muito maiores (Abdin *et al.*, 2024).

Abdin *et al.* (2024) ainda afirma que, por seu tamanho relativamente compacto, o phi3-mini pode ser executado de maneira local em um *smartphone* moderno, consumindo cerca de 1,8 GB de memória e rodando nativamente em um iPhone 14 com chip A16 Bionic, conforme relatado no trabalho. O phi3-mini, ainda, pode ser inicializado considerando o valor máximo de 32064 tokens, permitindo, assim, lidar com até mais que 32 mil “pacotes” de informação (Abdin *et al.*, 2024).

Além do modelo necessitar ter um número de parâmetros reduzido para viabilizar o seu uso em dispositivos locais, também é necessário garantir que se tenha configurado um ambiente capaz de lidar com a complexidade, as características e a otimização de processamento destes modelos de LLM, fora de um ambiente de computação de alto desempenho. Para isso, o *framework* de código aberto Ollama vem ganhando destaque, sobretudo pela sua agilidade de implementação e facilidade com configuração e utilização de modelos de LLM em ambiente local (Liu; Kang & Han, 2024). Através do Ollama, o modelo Phi3-Mini pode então ser rapidamente configurado, disponibilizado e utilizado, sem necessidade de recorrer a servidores em nuvem de processamento de alto desempenho.

2.4 *Prompt Engineering*

Liu et al. (2023) define *prompt* como sendo um conjunto de instruções fornecidas a um LLM, que permite o fornecimento de respostas personalizadas e aprimoradas dadas por

estes modelos. Ainda, o *prompt* pode ser visto como uma metodologia de programação e controle de qualidade para um modelo de LLM, fazendo, assim, a qualidade de um *prompt* influenciar na qualidade da resposta gerada pelo modelo (Liu *et al.*, 2023).

Prompt Enginnering, ou engenharia de prompts, é um processo pelo qual se busca garantir uma interação mais eficaz com modelos de LLM (White *et al.*, 2023). Com técnicas de *Prompt Enginnering* são analisadas alternativas que permitem uma maior eficiência na programação de LLMs, fornecendo regras, padrões de formatação das respostas esperadas, contexto da conversa, que informações são relevantes, dentre outros padrões (Liu *et al.*, 2023).

Assim, *Prompt Enginnering* é um campo que estuda as características e padrões que tendem a gerar uma maior qualidade ou um melhor direcionamento para respostas dadas por modelos de LLM (Liu *et al.*, 2023). Fazem parte de técnicas de *Prompt Enginnering* abordagens voltadas para maximizar a clareza e a objetividade de *prompts*, definir ao modelo a intenção e o contexto em que ele está situado, a motivação por trás do problema que está sendo resolvido, delimitar o formato do que se espera receber, bem como dar definições claras para parâmetros e informações que estão sendo considerados (White *et al.*, 2023).

2.5 Linguagem de Programação Python e Programação Orientada a Objetos

Python é uma linguagem de programação de sintaxe simples e de alta versatilidade, que permite a utilização de uma série de paradigmas de programação, dentre eles a programação funcional, caracterizada pelo reaproveitamento de blocos de códigos funcionais e a legibilidade de código, bem como a programação orientada a objetos (POO), que permite estruturar classes, que possuem características e variáveis compartilhadas, dando um nível de controle e customização muito avançado para desenvolvedores de software (Ogli, 2024A).

Especialmente, o relacionamento de classes e objetos de maneira simplificada, oriundo dos conceitos de POO, simplificam o gerenciamento e a implantação de sistemas complexos, modularizando o processamento em etapas, gerenciando estados e compartilhando variáveis. Toda essa versatilidade torna a linguagem Python altamente popular, o que atraiu uma vasta comunidade de desenvolvimento de código aberto, criando um ecossistema de bibliotecas nos mais variados segmentos, desde o desenvolvimento web até a inteligência artificial (Ogli, 2024A).

No contexto específico da IA, bibliotecas como TensorFlow, PyTorch, Scikit-Learn, Keras, dentre outras, oferecem uma série de funcionalidades avançadas, disponíveis de forma simples, acessíveis através da linguagem Python (Ogli, 2024B). Desta maneira, aproveitando recursos de paradigmas de programação sofisticados, com uma enorme quantidade de bibliotecas disponíveis, a seleção de Python como linguagem principal em projetos envolvendo IA torna-se uma seleção altamente promissora (Ogli, 2024B).

2.6 Noções Gerais: *Startups*, Planejamento Estratégico e Abordagem *Business Strategy Engineering* (BSE)

De acordo com Bortolini et al. (2021), startups podem ser definidas como organizações criadas para projetar e levar ao mercado produtos ou serviços competitivos, capazes de atender às necessidades dos clientes e gerar receita. Para o mercado, startups se destacam como organizações flexíveis, essenciais para promover a inovação, gerando vantagens competitivas em relação a empresas já estabelecidas (Bortolini *et al.*, 2021).

Durante o ciclo de vida de uma *startup*, ela pode ser identificada em fases características, normalmente definidas na seguinte ordem evolutiva: (I) Ideação, (II) Validação, (III) Operação, (IV) Tração e (V) Scale-Up (Da Silva; Bueno & De Souza, 2023). A fase (I), ideação, é caracterizada por uma fase inicial, caracterizada por pesquisa e/ou prototipagem; a fase (II), validação, é uma fase intermediária, de ajuste e melhoria, para entrar na etapa (III) a seguir, com o negócio já em operação. Na fase (IV), tração, a *startup* se encontra numa fase de expansão, tendo capacidade de tracionar seu próprio crescimento, e, por fim, na fase (V) uma *startup* atinge o *status quo* previsto para este modelo de negócios, que é uma fase de escala acelerada e elevadas proporções (Da Silva; Bueno & De Souza, 2023).

Especificamente em startups tecnológicas, que operam em um ambiente de rápida mudança, a adaptação e a resposta ágil a novos desafios é crucial para a sobrevivência e a competitividade no mercado. Para isso, é necessária a identificação de fatores-chave relevantes para o sucesso do negócio, para que as *startups* possam gerenciar bem os seus recursos, competências e capacidades de forma a obter resultados mais significativos no mercado (Yiu; Yeung & Jong, 2020). Para lidar com esta complexidade gerencial, é necessária uma harmonização entre a estratégia organizacional, a estrutura e a cultura da empresa com o seu respectivo desempenho (Zivojinovic, 2008).

Chiavenato (2009) define a estratégia organizacional como o processo de seleção de opções não-triviais, analisadas a partir de uma série de hipóteses existentes e escolhidas levando em conta aspectos internos e externos à organização. Gerenciar de forma estratégica, consiste, assim, em arquitetar processos e gerenciá-los de forma ampla, direcionando o modelo e o desempenho operacional da empresa à obtenção de seus objetivos (Chiavenato, 2009).

O planejamento estratégico, em concordância com Ribas et al. (2014), inicia com uma revisão da missão e visão organizacional, que identificam o que a empresa faz (missão) e o que ela projeta para o seu futuro (visão). A estratégia organizacional combina, assim, o entendimento conjunto destes fatores, contemplando ainda as propostas de valor entregues aos clientes, as vantagens competitivas que diferenciam a empresa frente aos seus concorrentes, os interesses dos clientes, combinando recursos e desafios internos e externos e definindo, assim, objetivos estratégicos: um conjunto de objetivos que tracionam uma empresa em direção à sua visão de futuro (Ribas *et al.*, 2014).

Desta maneira, a definição de objetivos estratégicos é uma abordagem naturalmente complexa e de um contexto amplo, que envolve diferentes agentes, informações e incertezas, internas e externas à organização. De modo a estruturar o processo de definição de objetivos estratégicos e manutenção de um planejamento estratégico em etapas, Medeiros (2016) apresentou a abordagem *Business Strategy Engineering* (BSE), conforme ilustração a seguir:

Figura 1 - O Método BSE em Etapas

Fonte: Medeiros (2023)

De maneira sintética, a abordagem consiste em uma etapa inicial de definição e análise de propostas de valor, seguindo por uma definição e revisão das vantagens competitivas para a *startup*, combinando informações internas e externas para, em etapa seguinte, definir os objetivos estratégicos empresariais (Medeiros, 2023). As etapas seguintes envolvem refino, implantação, monitoramento e melhoria contínua para o planejamento estratégico, que fogem do escopo do presente trabalho: que se limita, inicialmente, na etapa de definição dos objetivos estratégicos empresariais.

3 – METODOLOGIA

Neste capítulo são apresentados e discutidos os procedimentos aplicados, as ferramentas e abordagens utilizadas, o código desenvolvido e o procedimento de validação da prova de conceito realizada no desenvolvimento e execução deste trabalho. A metodologia aplicada neste trabalho foi uma análise exploratória e um estudo de caso direcionado, revisando funcionalidades gerais e validando, através de prova de conceito, o funcionamento geral da aplicação desenvolvida.

Para melhor organização deste capítulo, optou-se por dividir o processo em etapas, sendo elas: (I) Configuração de Ambiente e (II) Desenvolvimento de Código.

Todo o código fonte desenvolvido encontra-se disponível no Github, acessível através da URL: <https://github.com/gilvandrocesardemedeiros/AI-Assistant-for-Strategic-Planning>.

3.1 Configuração de Ambiente

Como parte dos desafios atrelados a este projeto, todo o processo de desenvolvimento efoi realizado em um *laptop* comercial de pequeno porte, com as seguintes especificações:

Tabela 1 - Configurações de dispositivo utilizado no trabalho

Modelo:	Laptop Dell Inspiron 15 5510
Processador:	Intel Core i7 - 11ª geração
Memória RAM:	16 GB
Placa de Vídeo:	Integrada Intel Iris Xe
Sistema Operacional:	Ubuntu 24.04.1 LTS

Fonte: Produzido pelo autor.

Além disso, o *framework* Ollama foi baixado e instalado via shell script (sh), com o comando: *curl -fsSL https://ollama.com/install.sh | sh*. Seguindo o passo a passo de instalação, ao final, a aplicação já estava pronta para uso. Com ela, a configuração do modelo Phi3-Mini se tornou altamente simples, bastando executar, via terminal, o comando: *ollama run phi3:mini*.

Com isso, através do Ollama, o modelo passou a estar disponível para a utilização no ambiente local, para a sua utilização. Necessitou-se, assim, configurar uma aplicação *client*, uma aplicação que permitisse se conectar ao modelo e interagir com ele. Para isso, foi instalada a aplicação de código aberto *openai*, através do comando *pip install openai*, disponibilizando assim a aplicação para estar disponível em ambiente de programação local Python.

Assim, foram definidas as variáveis MODEL_NAME, que referencia o nome adotado pelo Ollama para identificar o modelo phi3:mini, e, a partir da biblioteca openai, foi configurado o client para a interação com o modelo, com configuração a seguir ilustrada:

```
MODEL_NAME = "phi3:mini"

# Configura o client para o Ollama
client = openai.OpenAI(
   base_url="http://localhost:11434/v1",
   api_key="nokeyneeded"
)
```

Observa-se que a base_url trata-se de um endereço local (localhost), URL na qual o Ollama disponibilizou o modelo phi3:mini, e por se tratar de uma requisição local, sem envolver a API da empresa OpenAI, não foi necessária a definição de uma api_key, adotando, por default, a definição desta variável como sendo a string "nokeyneeded", reforçando que não há necessidade de qualquer conexão externa ao computador. A partir daqui, MODEL_NAME e client se tornam variáveis que passam a fazer parte da inicialização da aplicação principal, detalhada na seção a seguir.

3.2 Desenvolvimento de Código

O código desenvolvido consistiu principalmente de uma aplicação Python backend, isto é, um código modular, estruturado em uma classe nomeada por StartupStrategicPlanning, sendo esta classe responsável por definir métodos e controlar variáveis relativas ao processo

de obtenção de objetivos estratégicos, seguindo a linha de raciocínio proposta pela abordagem Business Strategy Engineering (Medeiros, 2023). A implementação do método de inicialização (__init__) da classe está apresentada a seguir:

```
class StartupStrategicPlanning:
    def __init__(self, client, model_name, startup_name, mission = "",
vision = "", customers = "", startup_stage = "", value_proposition =
"", competitive_advantages = "", performance = 1):
        """
Inicializa a classe com as informações da startup e configurações do
modelo.
        Args:
        - client: Cliente OpenAI configurado.
        - model_name: Nome do modelo a ser usado.
        - startup_name: Nome da startup.
        - Outros campos: Informações sobre a startup.
            - performance: Controle de performance (1 - mínima a 150 -
máxima). Default é 1.
        """
       start_time = time.time()
       dt_string = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
              self.LOG_FILENAME = dt_string + "_" + startup_name +
"_execution_log.txt"

       self.client = client
       self.model_name = model_name
       self.startup_info = {
            "startup_name": startup_name,
            "mission": mission,
            "vision": vision,
            "customers": customers,
            "startup_stage": startup_stage,
            "value_proposition": value_proposition,
            "competitive_advantages": competitive_advantages
        }

        self.strategic_objectives = ""

        self.refined_strategic_objectives = ""

        self.missing_informations = [
```

```
            key for key, value in self.startup_info.items() if value ==
""
        ]
        self.performance = performance

        end_time = time.time()

        # Monta um dicionário de inputs recebidos
        inputs = {
            "client": str(client),
            "model_name": model_name,
            "startup_name": startup_name,
            "mission": mission,
            "vision": vision,
            "customers": customers,
            "startup_stage": startup_stage,
            "value_proposition": value_proposition,
            "competitive_advantages": competitive_advantages,
            "performance": performance
        }
```

Assim, o processo de inicialização consiste no mapeamento das informações conhecidas, dentre elas as variáveis MODEL_NAME e client definidas na etapa anterior, e começando a agrupar alguns inputs que serão fornecidos pelo usuário, dentre eles a missão da startup, visão, vantagens competitivas, dentre outros. Estas informações não são obrigatórias, de modo que, na ausência de suas definições, o modelo phi3:mini fará o seu preenchimento, conforme detalhado a seguir.

O controle das variáveis fornecidas para o modelo nesta primeira etapa podem ser entendidas como as informações para formatar a base dos objetivos estratégicos que são ao final obtidos é feito através da variável self.startup_info, que passa então a ser utilizada nas etapas subsequentes.

As variáveis self.strategic_objectives e self.refined_strategic_objectives são inicializadas como um espaço em branco ("""), só sendo definidas nas etapas finais da classe desenvolvida. A variável self.missing_informations trata-se de uma lista, que armazena especificamente quais campos (isto é, quais informações) não foram fornecidas pelo usuário para a aplicação, e são a seguir preenchidas pelo próprio modelo de IA, considerando o contexto e as demais informações fornecidas.

A variável performance, que foi definida pelo usuário e armazenada no método de inicialização da classe StartupStrategicPlanning, é uma variável que controla o número de tokens utilizados pelo modelo de SLM, sendo adotado, por default, o valor “1” (que seria o mínimo de tokens, o que se espera ser mais eficiente). Este processo é melhor detalhado nas etapas subsequentes, na definição do modelo.

Além disso, para supervisionar todo o código, foi definido um método log_operation, utilizado já na inicialização da classe e reutilizado em etapas subsequentes, que se caracteriza por um método que faz o registro do tempo de processamento de cada método chamado e das operações que foram realizadas, de modo a ter um acompanhamento passo a passo da execução da aplicação desenvolvida. A implementação deste método é um processo puramente de mapeamento de informações em código, não sendo útil a sua explicação no corpo deste trabalho. Para acessar a sua definição exata, o repositório no Github fornecido ao início deste capítulo pode auxiliar.

A seguir, o método get_max_tokens ilustra o processo de conversão entre o valor definido para a variável self.performance e sua respectiva conversão em número máximo de tokens utilizados pelo modelo. Como comentado anterior, o número máximo de tokens para o modelo Phi3-Mini trata-se de 32064 tokens, e o valor max_t é definido de modo que, o valor mínimo para a variável “performance”, isto é, o valor 1, resulte em um número máximo de tokens de 200 (valor arbitrariamente selecionado), e o valor máximo sugerido para “performance”, que seria 150, seja convertido a um valor próximo do máximo de tokens que o modelo trabalha, o valor de 30000 tokens. Assim, tornou-se possível controlar de maneira sintética a performance do modelo, bastando controlar uma variável numérica e simples, chamada de “performance”. A implementação em mais detalhes pode ser vista abaixo.

```
def get_max_tokens(self): #Max tokens - default: 32064 tokens
        """Calcula o limite de tokens com base na performance."""
            max_t = 200 + (self.performance - 1) * 200 # 200 para
performance 1, até 30000 para performance 150
        return max_t
```

Ainda, foi definido um método get_definition, que fornece para o modelo, sempre que necessário, as definições que foram adotadas para cada parâmetro que está sendo utilizado

para fundamentar a linha de raciocínio para a elaboração dos objetivos estratégicos. Este método adotou a seguinte convenção de definições apresentadas na Tabela a seguir:

Tabela 2 - Definições adotadas para melhor direcionar o modelo

Nome da Startup	"Um nome comercial competitivo, que ganhe destaque no mercado."
Missão	"A missão define o propósito principal da startup, ou seja, o que ela faz e para quem."
Visão	"A visão é uma representação objetiva do futuro esperado para a startup."
Clientes	"Os clientes representam o público-alvo ou mercado para o qual a startup oferece seus produtos ou serviços."
Estágio da Startup	"O estágio da startup refere-se à fase do negócio, podendo ser, em ordem de evolução da startup: Ideação, Validação, Operação, Tração ou Scale-Up."
Proposta de Valor	"As propostas de valor são os benefícios entregues pela empresa sob a ótica do cliente, o que agrega valor ao serviço ou produto entregue ao cliente."
Vantagens Competitivas	"As vantagens competitivas são os diferenciais que destacam a startup frente aos concorrentes no mercado."

Fonte: Produzido pelo autor.

A seguir, é apresentado um dos métodos mais importantes para a classe, o método model_predict, sendo este método utilizado para realizar a definição e a chamada do modelo Phi3-Mini.

```
def model_predict(self, prompt):
        """Instancia o modelo para processar o prompt."""
        messages = [
                {"role": "system", "content": "Você é um assistente de planejamento estratégico para uma startup. Você responde sempre de maneira objetiva e em língua portuguesa."},
            {"role": "user", "content": prompt}
        ]
        response_text = None

        try:
            response = self.client.chat.completions.create(
```

```
            model=self.model_name,
            messages=messages,
            max_tokens=self.get_max_tokens(),
            temperature=0.05
        )
        response_text = response.choices[0].message.content
    except Exception as e:
        response_text = None

    return response_text
```

Este método toma como base o recebimento de um prompt como input. O referido prompt é formatado por métodos auxiliares, discutidos abaixo, e serve para formatar melhor os objetivos, o contexto e os resultados esperados em cada etapa. O método model_predict define o formato geral da mensagem a ser enviada para o modelo Phi3-Mini, considerando o prompt recebido, mas definindo para todos os casos uma mesma inicialização padrão, que define para o modelo: “Você é um assistente de planejamento estratégico para uma startup. Você responde sempre de maneira objetiva e em língua portuguesa.”.

Com esta inicialização recorrente, o modelo entende melhor o contexto ao qual está inserido, sua motivação e objetivos, de modo assim a obter resultados mais precisos, refinados e eficientes, conforme sugerido pela literatura (White et al., 2023; Liu et al., 2023).

Em linhas gerais, além da inicialização do modelo, o método model_predict também realiza a chamada do modelo, fornecendo como inputs, além da referência do modelo (model_name), a mensagem formatada (combinando a inicialização padrão e o prompt fornecido), também fornece o número máximo de tokens que devem ser considerados, e, por fim, define o argumento temperature, que corresponde ao grau de “aleatoriedade” que o modelo irá considerar. De maneira interpretativa, um valor maior de temperature caracteriza respostas mais aleatórias, podendo ser, em alguns casos, mais criativas, mas podendo ser também mais distantes do que se espera receber. De maneira similar, um menor valor de temperature se caracteriza por respostas mais diretas, mais lógicas, se limitando ao escopo das informações para as quais o modelo está sendo utilizado. Para evitar respostas distorcidas, e avaliar melhor a influência da qualidade do prompt na resposta gerada pelo modelo, foi fornecido um valor arbitrário de 0.05 para este parâmetro, adotando a preferência de um valor muito próximo de 0 (que seria a mínima influência da aleatoriedade). Os métodos gerais,

discutidos acima, passam a ser então reutilizados nos métodos com objetivos específicos, discutidos a seguir.

O método fill_informations, como já comentado anteriormente, atribui ao modelo a capacidade de lidar com a ausência de informações dentre os inputs fornecidos pelo usuário, de modo, assim, a explorar possíveis características úteis para um assistente inteligente de planejamento estratégico. A implementação do método está exposta a seguir:

```
def fill_informations(self):
        """Preenche campos faltantes utilizando o modelo."""
        start_time = time.time()

        for key, value in self.startup_info.items():
            if value == "":
                        prompt = f"""Aqui estão as informações atuais da
startup: {self.format_dict_as_table(self.startup_info, sep = ",")}""" +
\
                  f"""\nConsidere que a definição para o campo "{key}" é:
{self.get_definition(key)}""" + \
                 f"""\nPreciso que você forneça uma resposta objetiva que
possa ser usada no campo "{key}"."""

                response = self.model_predict(prompt)

                self.startup_info[key] = response

        end_time = time.time()

        self.log_operation(
            function_name="fill_informations",
            inputs={},
            results=self.format_dict_as_table(self.startup_info),
            start_time=start_time,
            end_time=end_time
        )
```

O principal ponto de interesse para este método é a definição do seu *prompt*, caracterizado pela junção de blocos de informações em sequência, seguindo a linha de raciocínio de, primeiro, apresentar ao modelo em que contexto ele está (“"Aqui estão as

informações atuais da startup [...]”), em seguida apresentando para ele as definições de cada um dos campos faltantes (isto é, as definições apresentadas na Tabela 2, acima), garantindo uma melhor compreensão do modelo do que se está considerando como resposta. Por fim, o prompt define um último direcionamento para a resposta, informando que “Preciso que você forneça uma resposta objetiva [...]”, que possa ser utilizada no referido campo, para o qual se foi dada a definição. Assim, o modelo pode identificar melhor o estágio em que está, com o que exatamente se está trabalhando e o que se pretende obter. O resultado desta etapa preenche lacunas que porventura não tenham sido preenchidas nas informações gerais, armazenadas na variável self.startup_info.

Em seguida, como uma forma de controle de qualidade, foi definido o método response_quality_control, que se caracteriza por um método de controle de qualidade baseado na análise do contexto das informações da startup. Isto é, caso a missão da empresa esteja desconexa ou inapropriada, o próprio modelo pode sugerir uma versão que julgue mais apropriada. Para este método, foi considerada a opção de controlar se o modelo pode aplicar controle de qualidade com base nos dados fornecidos originalmente pelo usuário, ou se deve aplicar controle de qualidade apenas nas respostas geradas por IA (assumindo, assim, que as informações fornecidas pelo usuário devem ser imutáveis).

O prompt estabelecido para este método consistiu em uma construção similar a outros já aqui apresentados, sendo composto por um encadeamento de informações, estruturados na Tabela a seguir:

Tabela 3 - Formato do prompt para o método *response_quality_control*

Objetivo	**Descrição textual - *prompt***
Contextualização geral	"Aqui estão as informações atuais da startup: [...]”
Escopo	“A resposta sugerida para o campo [...] foi: [...]"
Objetivo	“Revise e resuma a descrição da resposta, elimine informações desconexas ou inapropriadas.”
Formatação da resposta	“Retorne apenas a versão final do texto para ser utilizado no campo [...]”

Fonte: Produzido pelo autor.

Assim, iterando por cada elemento (campo), analisando o contexto geral (informações atuais da startup), o modelo consegue avaliar se a resposta está apropriada ou não, e caso não,

ele retorna uma versão alternativa para ser considerada, aplicando assim um controle de qualidade em cada campo que seja de interesse.

Foi definidos, também como métodos de controle de qualidade, o método *review_specific_response*, que permite ao usuário selecionar uma informação-alvo (um campo) que julgue inapropriado, pedindo uma nova iteração do modelo para este campo buscando a sua correção autônoma, e o método *review_target_with_comment*, que permite ao usuário informar um campo que julgue incorreto ou inapropriado e fornecer um comentário, um tipo de *feedback*, para ser considerado pelo modelo no momento de gerar uma nova interpretação. Os *prompts* formatados para cada um destes métodos são apresentados a seguir:

Tabela 4 - Formato do prompt para o método *review_specific_response*

Objetivo	**Descrição textual - *prompt***
Contextualização geral	"Aqui estão as informações atuais da startup: [...]”
Escopo	“A resposta sugerida para o campo [...] foi: [...]"
Objetivo	“Revise e resuma a descrição da resposta, elimine informações desconexas ou inapropriadas.”
Formatação da resposta	“Retorne apenas a versão final do texto para ser utilizado no campo [...]”

Fonte: Produzido pelo autor.

Como se pode observar acima, o *prompt* definido para o método *review_specific_response* não se diferencia do *prompt* antes aplicado pelo método *response_quality_control*. Este método adicional permite, basicamente, solicitar uma nova iteração autônoma do modelo para obter uma resposta para uma dada informação. Como o processo interpretativo de um modelo de LLM não é determinístico, iterações realizadas considerando inputs idênticos, podem gerar respostas diferentes. Assim, o método *review_specific_response* se caracteriza como uma simples revisão, na busca de se obter uma resposta diferente, que possa ser mais satisfatória.

Embora exista a chance do modelo obter melhores resultados sozinho, a tendência é que informações adicionais ou comentários o façam performar melhor. Assim, o método *review_target_with_comment* habilita esta possibilidade, com um *prompt* estruturado conforme se mostra a seguir:

Tabela 5 - Formato do prompt para o método *review_target_with_comment*

Objetivo	**Descrição textual - *prompt***
Contextualização geral	"Aqui estão as informações atuais da startup: [...]”
Escopo	“O campo [...] precisa ser melhorado."
Direcionamento	“O seguinte comentário foi feito: [...]”
Objetivo	"Revise o conteúdo do campo e sugira uma versão aprimorada com base no comentário."

Fonte: Produzido pelo autor.

Assim, com as iterações e métodos acima, são feitas definições e aplicados controles de qualidade nas informações gerais da startup e, a partir delas, são realizadas as definições dos objetivos estratégicos, com os métodos discutidos a seguir.

O método *generate_strategic_objectives* é a primeira etapa na definição de objetivos estratégicos. Para o seu funcionamento, as informações disponíveis para a *startup* foram agrupadas em pares, dividindo o problema em etapas, e, ao final, integrando todas as informações: isto é, permitindo que o modelo possa gerar uma série de objetivos estratégicos olhando para cada par de informações inicialmente, e ao final, elaborando objetivos estratégicos em uma visão macro, com todas as informações sendo fornecidas ao modelo.

Assim, os pares de informações formadas foram:

- Par 1: Missão e Visão
- Par 2: Clientes e Fase da *Startup* (Ideação, Validação, Operação, Tração ou Scale-Up)
- Par 3: Proposta de Valor e Vantagens Competitivas

O *prompt* formatado para este método consistiu no formato apresentado na Tabela a seguir:

Tabela 6 - Formato do prompt para o método *generate_strategic_objectives*

Objetivo	**Descrição textual - *prompt***
Contextualização geral	"Considere as seguintes informações relativas à minha startup: [...]”
Escopo	“Gere uma lista de objetivos estratégicos para a startup considerando estas informações."
Direcionamento	“Cada objetivo estratégico deverá ser uma descrição curta e clara com base nas informações fornecidas.”
Formato da resposta	"Cada item da lista deverá ser separado por ponto e vírgula - caractere ';' "

Fonte: Produzido pelo autor.

As informações fornecidas para o modelo a cada iteração corresponderam a iterar sobre cada um dos pares acima definidos, um par por vez. Ao final, todas as informações são disponibilizadas em conjunto, totalizando 4 iterações (3 por pares e 1 iteração global). Ao fim do processamento, uma lista de objetivos estratégicos é então armazenada na variável *strategic_objectives*.

Novamente direcionando esforços em melhoria contínua, foi definido o método *refine_strategic_objectives*, que não só revisa a estrutura dos objetivos estratégicos definidos na etapa anterior, mas também orienta o modelo na busca do refino e resumo de cada objetivo isoladamente. O formato do *prompt* definido consistiu no formato detalhado na Tabela abaixo:

Tabela 7 - Formato do prompt para o método *refine_strategic_objectives*

Objetivo	**Descrição textual - *prompt***
Contextualização geral	"Aqui estão os objetivos estratégicos atuais da minha startup, separados por ponto e vírgula: [...]”
Escopo	"Revise e torne cada objetivo o mais consistente e resumido possível,”
Direcionamento	“Cada objetivo estratégico deverá ser uma descrição curta e clara com base nas informações fornecidas,”
Formato da resposta	"mantendo cada um separado por ponto e vírgula. Retorne o texto final com os objetivos refinados."

Fonte: Produzido pelo autor.

Desta maneira, o modelo tende a retornar respostas mais bem formatadas, podendo otimizar a qualidade das respostas no mesmo passo.

Por fim, visando um direcionamento extra para a classe desenvolvida, foi definido o método *priority_objectives*, que consiste na análise dos objetivos estratégicos já delimitados, comparando-os de volta com base nas informações gerais da startup, analisando sua importância e retornando os objetivos por ordem de prioridade. O formato do *prompt* fornecido consistiu na Tabela a seguir apresentada:

Tabela 8 - Formato do prompt para o método *priority_objectives*

Objetivo	**Descrição textual - *prompt***
Contextualização geral	"Aqui estão as informações da minha startup: [...] Aqui está a lista de objetivos estratégicos, separados por ponto e vírgula: [...]"
Escopo e direcionamento	"Priorize esses objetivos do mais importante ao menos importante, considerando as informações da startup"
Formato da resposta	"Retorne cada objetivo separado por ponto e vírgula. O primeiro deve ser o mais prioritário."

Fonte: Produzido pelo autor.

Desta maneira, foi concluída a linha de raciocínio para definição e revisão de objetivos estratégicos, em um processo assistido por IA.

No capítulo posterior, é feita uma demonstração da utilização da aplicação desenvolvida através de um Jupyter-Notebook, ilustrando o passo a passo em um estudo de caso hipotético. É feita também uma análise de tempo de processamento, bem como uma análise comparativa das respostas e do tempo de processamento, mantendo fixos os inputs da classe e variando apenas o parâmetro *performance*.

4 – PROVA DE CONCEITO, RESULTADOS E DISCUSSÕES

Para avaliar a aplicação desenvolvida, foram realizados alguns estudos de caso como prova de conceito, utilizando dados hipotéticos, que estão também disponíveis no repositório do Github, através de um Jupyter-Notebook. Em cada estudo de caso, foi considerado como base a *startup* fictícia "DisnAI", uma ideia de negócios, para a qual foram definidas as seguintes informações na inicialização da classe:

Figura 2 - Inicialização da classe *StartupStrategicPlanning* no estudo de caso realizado

```
# Instancia o assistente de estratégia
startup_assistant = StartupStrategicPlanning(
    client=client,
    model_name=MODEL_NAME,
    startup_name="DisnAI",
    mission="Promover uma fonte de entretenimento participativa, no qual o usuário se torna também parte da construção do enredo",
    vision="",
    customers="Usuários residenciais, pessoas comuns",
    startup_stage="Ideação",
    value_proposition="",
    competitive_advantages="Uso de modelos generativos avançados",
    performance = 1
)
```

Fonte: Produzido pelo autor.

Propositalmente, alguns campos foram deixados em branco, a saber: *vision* (visão) e *value_proposition* (proposta de valor). Em seguida, cada um dos métodos desenvolvidos foi chamado, conforme ilustrado a seguir. O método *log_operation*, pertencente a cada um destes métodos, monitora as operações realizadas e o tempo decorrido para a realização de cada operação, salvando estas informações em um arquivo .txt local, utilizado para fins de monitoramento das operações realizadas pelo modelo.

Nas seções a seguir, é apresentado um estudo de caso completo e detalhado, considerando mínima performance estabelecida (valor do parâmetro performance = 1), e posteriormente são discutidos de forma comparativa as diferenças observadas nos resultados e no tempo de processamento decorrente da variação deste parâmetro para os valores 10 e 100.

4.1 Estudo de Caso Completo - Modelo em Mínima Performance

No primeiro estudo de caso completo realizado, foi definida a menor performance trabalhada, com valor 1, que se converte em 200 *tokens* utilizados pelo modelo. Este parâmetro é utilizado ao longo de todo o processo, e os resultados são a seguir apresentados.

Após inicialização da classe, foi aplicado o *pipeline* de processamento em etapas lógicas, estruturadas conforme apresentado a seguir:

Figura 3 - Rotina completa de utilização do modelo e dos métodos desenvolvidos

```
[5]: # Preenche os campos ausentes usando o modelo
startup_assistant.fill_informations()

[6]: # Verifica e aplica controle de qualidade sem considerar respostas originais
startup_assistant.response_quality_control(include_original_responses = False)

[7]: # Verifica e aplica controle de qualidade considerando respostas originais
startup_assistant.response_quality_control(include_original_responses = True)

[8]: # Revisa um tópico específico sob demanda
startup_assistant.review_specific_response("competitive_advantages")

[9]: # Revisa um campo específico com comentário adicional
startup_assistant.review_target_with_comment(target = "vision", comment = "Minha visão não estabelece uma meta tangível para os próximos anos

[10]: # Define objetivos estratégicos com base nas informações disponíveis para a startup
startup_assistant.generate_strategic_objectives()

[11]: # Revisa objetivos estratégicos com base nas informações disponíveis para a startup
startup_assistant.refine_strategic_objectives()

[12]: # Ranqueia os objetivos estratégicos do mais importante ao menos importante para a startup
startup_assistant.priority_objectives()
```

Fonte: Produzido pelo autor.

Em linhas gerais, o método *fill_informations*() foi utilizado para preencher as informações de “visão” e “proposta de valor”, obtendo os resultados ilustrados na tabela a seguir. São destacados em negritos os campos que passaram por atualização pelo modelo, sendo adotado este padrão para as demais tabelas.

Tabela 9 - Informações básicas da startup após chamada do método *fill_informations*

Nome da startup	DisnAI
Missão	Promover uma fonte de entretenimento participativa, no qual o usuário se torna também parte da construção do enredo
Visão	**Estabelecer-se como um dos principais players na indústria do entretenimento participativo, onde a tecnologia de IA não apenas personaliza o conteúdo para cada usuário as necessidades e preferências através de modelos generativos avançados. A DisnAI visa criar narrativas dinâmicas que se adaptam ao feedback dos usuários em tempo real, permitindo uma experiência única e imersiva onde o público não é apenas um espectador passivo, mas também contribui ativamente para a construção do enredo. A DisnAI pretende expandir seu alcance além de residentes urbanos para incluir pessoas comuns em todo o mundo, tornando-se uma plataforma global que promove a criatividade e engajamento coletivo no entretenimento digital.**
Clientes	Usuários residenciais, pessoas comuns
Estágio da Startup	Ideação

Proposta de Valor	**A DisnAI oferece aos seus usuários a oportunidade de participar ativamente na criação e desenvolvimento do conteúdo, proporcionando-lhes uma experiência única que vai além da mera visualização. Ao utilizar modelos generativos avançados para personalizar narrativas em tempo real com base no feedback dos usuários, a DisnAI garante que cada indivíduo se sinta diretamente envolvido na construção do enredo e da história sendo contada. Essa interação dinâmica não apenas melhora o entretenimento ao torná-lo mais imersivo e engajador, mas também promove a criatividade individual e coletiva em um ambiente digital globalizado. Além disso, expandindo seu alcance além de residentes urbanos para incluir pessoas comuns por**
Vantagens Competitivas	Uso de modelos generativos avançados

Fonte: Produzido pelo autor.

Observa-se que, em linhas gerais, o modelo não seguiu bem as orientações pedindo respostas resumidas, como especificado via prompt, nesta etapa. Ainda, observa-se que ele não conseguiu concluir uma definição para a proposta de valor, encerrando a resposta de maneira incompleta, deixando em aberto a conclusão. Esta operação levou 67,21 segundos ao todo para ser realizada.

Em seguida, utilizando o método *response_quality_control*, inicialmente revisando apenas as respostas dadas pelo modelo de IA (setando o parâmetro "include_original_responses" com valor "False"), o modelo apresentou as seguintes respostas:

Tabela 10 - Informações da startup revisadas pelo método *response_quality_control* - Etapa 1

Nome da startup	DisnAI
Missão	Promover uma fonte de entretenimento participativa, no qual o usuário se torna também parte da construção do enredo
Visão	**Estabelecer-se como um dos principais players na indústria do entretenimento participativo, onde a tecnologia de IA personaliza conteúdo em tempo real com base no feedback dos usuários. A DisnAI visa criar narrativas dinâmicas que se adaptam às necessidades e preferências individuais através de modelos generativos avançados, promovendo uma experiência única e imersiva onde o público contribui ativamente para a construção do enredo. A DisnAI pretende expandir seu alcance além dos residentes urbanos para incluir pessoas comuns em todo o mundo, tornando-se uma plataforma global que promove criatividade e engajamento coletivo no entretenimento digital.**

Clientes	Usuários residenciais, pessoas comuns
Estágio da Startup	Ideação
Proposta de Valor	**A DisnAI oferece aos seus usuários uma experiência única de criação e desenvolvimento ativo, onde cada indivíduo se sente diretamente envolvido na construção do enredo. Utilizando modelos generativos avançados para personalizar narrativas em tempo real com base no feedback dos usuários, a DisnAI melhora o entretenimento tornando-o mais imersivo e engajador, ao mesmo tempo que promove criatividade individual e coletiva. Além disso, expandindo seu alcance para incluir pessoas comuns além de residentes urbanos, a DisnAI se destaca como uma plataforma globalizada no entretenimento digital participativo.**
Vantagens Competitivas	Uso de modelos generativos avançados

Fonte: Produzido pelo autor.

As modificações realizadas em “Visão” foram principalmente vistas em termos de pequenas reduções textuais e uma seleção de palavras alternativas, trocando, por exemplo, a sentença “permitindo uma experiência única e imersiva” pela sentença “promovendo uma experiência única e imersiva”.

Para a “Proposta de Valor” o resultado foi mais perceptível. Foram observados, por exemplo, que o texto se tornou significativamente menor, mais objetivo, e a problemática principal vista no passo anterior, decorrente da resposta estar incompleta, foi resolvida, sendo esta versão final uma versão com conclusão. No processamento desta etapa, levou-se um tempo de 91,71 s.

Na etapa seguinte, foi utilizado novamente o método *response_quality_control*, mas permitindo agora que ele alterasse, também, as informações originalmente fornecidas para o modelo (setando o parâmetro “include_original_responses” com valor “True”), obtendo a resposta a seguir:

Tabela 11 - Informações da startup revisadas pelo método *response_quality_control* - Etapa 2

Nome da startup	DisnAI
Missão	**Promover uma fonte de entretenimento participativa onde o usuário se torna parte da construção do enredo, personalizada em tempo real com base no feedback dos usuários.**

Visão	Estabelecer-se como um dos principais players na indústria do entretenimento participativo, onde a tecnologia de IA personaliza conteúdo em tempo real com base no feedback dos usuários. A DisnAI visa criar narrativas dinâmicas que se adaptam às necessidades e preferências individuais através de modelos generativos avançados, promovendo uma experiência única e imersiva onde o público contribui ativamente para a construção do enredo. A DisnAI pretende expandir seu alcance além dos residentes urbanos para incluir pessoas comuns em todo o mundo, tornando-se uma plataforma global que promove criatividade e engajamento coletivo no entretenimento digital.
Clientes	**Usuários residentes urbanos e pessoas comuns em todo o mundo.**
Estágio da Startup	**A DisnAI está em uma fase de ideação com o objetivo de estabelecer-se como um dos principais players na indústria do entretenimento participativo. A startup visa criar narrativas dinâmicas que se adaptam às necessidades e preferências individuais, promovendo experiências únicas e imersivas onde o público contribui ativamente para a construção do enredo. Além disso, pretende expandir seu alcance além dos residentes urbanos para incluir pessoas comuns em todo o mundo, tornando-se uma plataforma globalizada no entretenimento digital participativo.**
Proposta de Valor	A DisnAI oferece aos seus usuários uma experiência única de criação e desenvolvimento ativo, onde cada indivíduo se sente diretamente envolvido na construção do enredo. Utilizando modelos generativos avançados para personalizar narrativas em tempo real com base no feedback dos usuários, a DisnAI melhora o entretenimento tornando-o mais imersivo e engajador, ao mesmo tempo que promove criatividade individual e coletiva. Além disso, expandindo seu alcance para incluir pessoas comuns além de residentes urbanos, a DisnAI se destaca como uma plataforma globalizada no entretenimento digital participativo.
Vantagens Competitivas	**A DisnAI utiliza modelos generativos avançados, que permitem personalizar narrativas em tempo real com base no feedback dos usuários. Isso melhora o entretenimento tornando-o mais imersivo e engajador, ao mesmo tempo que promove criatividade individual e coletiva.**

Fonte: Produzido pelo autor.

Observa-se que, nesta etapa, o modelo optou por não realizar alterações na “Visão” e na “Proposta de Valor”, mantendo-as conforme apresentado na etapa anterior, mas realizou mudanças em informações originalmente fornecidas. Na missão, por exemplo, ele optou por trocar a versão original “Promover uma fonte de entretenimento participativa, no qual o usuário se torna também parte da construção do enredo” por uma versão mais descritiva: “Promover uma fonte de entretenimento participativa onde o usuário se torna parte da

construção do enredo, personalizada em tempo real com base no feedback dos usuários". De maneira considerada inteligente, o modelo adicionou também na missão a informação de que a empresa permite uma personalização "em tempo real com base no feedback dos usuários", que antes não fazia parte da missão da empresa - mas, pelo contexto, faz sentido.

Em "Clientes" e em "Vantagens Competitivas", o modelo optou por descrever melhor as informações, considerando o contexto. Embora não sigam a orientação de serem informações resumidas, apresentou-se um valor agregado considerável para estas informações. No entanto, especificamente para o campo "Estágio da Startup", o modelo não conseguiu entender que esta informação é categórica, e preferiu transformá-la em um texto descrevendo o entendimento para a fase de "Ideação", não apenas a informação "Ideação", como antes informado.

Embora estivesse implícito, na definição do parâmetro "Estágio da Startup" que o significado dele seria "Ideação, Validação, Operação, Tração ou Scale-Up", a falta de especificação exata de que esta informação precisa ser categórica fez o modelo acreditar que este campo poderia ser melhor descrito, e assim o fez. A execução completa desta etapa levou 170,79 segundos.

Em seguida, de maneira arbitrária, foi solicitado ao modelo revisar a resposta específica dada ao campo "Vantagens Competitivas", utilizando o método *review_specific_response*. Embora o modelo tenha processado a informação, levando 21,85 segundos de processamento, ele optou por não realizar alterações, mantendo-a como exposto na etapa anterior. Acredita-se que a decisão por estabelecer o parâmetro *temperature* com um valor muito próximo de 0, mais precisamente 0.05, fez com que o modelo tendesse a oscilar menos nas respostas geradas, com menor aleatoriedade - o que pode ser entendido, também, como uma "menor criatividade".

Na próxima etapa, foi utilizado o método *review_target_with_comment*, solicitando a revisão do campo "Visão" com o comentário: "'Minha visão não estabelece uma meta tangível para os próximos anos''. A resposta obtida pelo modelo foi significativamente extensa, apresentada de maneira comparativa com a versão anterior na tabela abaixo:

Tabela 12 - Modificações realizadas pelo modelo no campo de “Visão”

Visão Anterior	Visão Revisada
Estabelecer-se como um dos principais players na indústria do entretenimento participativo, onde a tecnologia de IA personaliza conteúdo em tempo real com base no feedback dos usuários. A DisnAI visa criar narrativas dinâmicas que se adaptam às necessidades e preferências individuais através de modelos generativos avançados, promovendo uma experiência única e imersiva onde o público contribui ativamente para a construção do enredo. A DisnAI pretende expandir seu alcance além dos residentes urbanos para incluir pessoas comuns em todo o mundo, tornando-se uma plataforma global que promove criatividade e engajamento coletivo no entretenimento digital.	A DisnAI tem como objetivo se consolidar na indústria do entretenimento participativo, tornando-se um dos principais players em pouco tempo. Com a implementação de modelos generativos avançados que personalizam conteúdo em tempo real com base no feedback direto dos usuários, estamos criando narrativas dinâmicas e adaptáveis às necessidades individuais. Essa abordagem inovadora promove experiências únicas e imersivas onde o público não é apenas um espectador passivo, mas se torna ativamente envolvido na construção do próprio enredo. A DisnAI pretende expandir seu alcance além dos residentes urbanos para incluir pessoas comuns em todo o mundo, tornando-se uma plataforma globalizada que promove

Fonte: Produzido pelo autor.

Observa-se que o modelo tentou elaborar uma versão mais ampla do texto atribuído à visão da empresa, mas não conseguiu concluí-la. Acredita-se que a limitação nos *tokens*, dado que este experimento foi inteiramente realizado com apenas 200 *tokens*, tenha gerado esta limitação. Embora tenha tentado atribuir uma meta para a empresa, com a frase “tornando-se um dos principais players em pouco tempo”, o modelo não conseguiu atribuir uma meta temporal - o que faz sentido, dado que ele não tem informações sobre a capacidade operacional, os recursos financeiros da startup ou os concorrentes diretos no mercado. O tempo de processamento nesta etapa foi de 50,47 segundos.

Na próxima etapa, partiu-se diretamente para a obtenção de objetivos estratégicos, utilizando o método *generate_strategic_objectives*, obtendo as respostas apresentadas a seguir:

Tabela 13 - Objetivos estratégicos obtidos na primeira iteração

Etapa	Objetivos Estratégicos
Par Missão e Visão	1. Estabelecer a DisnAI como um dos principais players na indústria do entretenimento participativo, expandindo o alcance para incluir pessoas comuns em todo o mundo; 2. Implementar modelos generativos avançados que personalizem conteúdo em tempo real baseado no feedback direto dos usuários, criando narrativas dinâmicas e adaptáveis às necessidades individuais; 3. Promover experiências únicas e imersivas onde o público se torna ativamente envolvido na construção do próprio enredo em vez de ser um espectador passivo; 4. Estabelecer a DisnAI como uma plataforma globalizada que oferece conteúdo personalizado para diversas culturas e idiomas, expandindo o alcance além dos residentes urbanos; 5.
Par Clientes e Estágio	1. Estabelecer-se como um dos principais players na indústria do entretenimento participativo, oferecendo narrativas dinâmicas que se adaptam às necessidades individuais; 2. Promover experiências únicas e imersivas onde o público contribui ativamente para a construção do enredo, incentivando engajamento e fidelidade ao produto; 3. Expandir seu alcance além dos residentes urbanos para incluir pessoas comuns em todo o mundo, tornando-se uma plataforma globalizada no entretenimento digital participativo; 4. Desenvolver parcerias estratégicas com empresas e influenciadores globais que possam ajudar na expansão da marca para novos mercados internacionais; 5. Investir em pesquisa de mercado contínua para
Par Proposta de Valor e Vantagem Competi-tiva	1. Aprimorar a experiência imersiva do usuário ao utilizar feedback para personalizar narrativas em tempo real; 2. Incentivar a criatividade individual e coletiva por meio da interação direta com o enredo construído pelos usuários; 3. Expandir o alcance globalizado, atraindo pessoas de diferentes regiões para participar do entretenimento digital; 4. Construir uma comunidade forte e comprometida que valorize a contribuição dos indivíduos na criação das narrativas; 5. Estabelecer parcerias com plataformas globais de mídia social para promover o DisnAI como um espaço único para entretenimento participativo; 6. Desenvolver novas tecnologias e modelos generativos que continuem a melhorar a experiência do us

Objetivos Combina-dos	1. Estabelecer DisnAI como um dos principais players na indústria do entretenimento participativo em pouco tempo; 2. Implementar modelos generativos avançados para personalizar conteúdo em tempo real com base no feedback direto dos usuários, criando narrativas dinâmicas e adaptáveis às necessidades individuais; 3. Promover experiências únicas e imersivas onde o público não é apenas um espectador passivo, mas se torna ativamente envolvido na construção do próprio enredo; 4. Expandir seu alcance além dos residentes urbanos para incluir pessoas comuns em todo o mundo, tornando-se uma plataforma globalizada no entretenimento digital participativo; 5. Melhorar a criatividade individual e coletiva ao permitir que os us

Fonte: Produzido pelo autor.

Como esta ainda não é a etapa final para os objetivos estratégicos, não é aqui analisada a qualidade das respostas. No entanto, observa-se que o padrão solicitado, de separar cada objetivo por ponto e vírgula (;) foi seguido à risca pelo modelo, mas também de maneria geral observa-se que ele não conseguiu concluir as respostas, obtendo respostas inconclusivas. O tempo de processamento desta etapa foi o maior de todos, levando 191,79 segundos para ser processado.

No passo seguinte, foi utilizado o método *refine_strategic_objectives*, que combina todos os resultados obtidos em uma resposta única, apresentada a seguir:

Tabela 14 - Objetivos estratégicos refinados

Objetivo	**Descrição**
1	Estabelecer DisnAI como líder no entretenimento participativo global através de narrativas personalizadas;
2	Utilizar modelos generativos avançados para criar conteúdo dinâmico e adaptável ao feedback dos usuários, promovendo a criação colaborativa do enredo;
3	Garantir que o público seja um ator-criador fundamental na experiência imersiva da plataforma DisnAI;
4	Estender a presença global de DisnAI além dos urbanos, alcançando pessoas comuns em todo mundo e diversificando culturas e idiomas no conteúdo oferecido;
5	Formar parcerias estratégicas para expandir o alcance internacional da marca DisnAI através de empresas globais e influenciadores;
6	Investir continuamente

Fonte: Produzido pelo autor.

Observou-se uma boa capacidade de sintetizar informações, gerando um agrupamento bem mais enxuto, mas ainda muito relacionado aos objetivos gerados originalmente. Notou-se, novamente, que o modelo não conseguiu concluir a escrita do Objetivo 6. Em uma análise superficial, os resultados fazem sentido e parecem úteis, considerando que boa parte destas informações nesta etapa já são criações de um modelo de IA configurado próximo às suas especificações mínimas. O tempo de processamento desta etapa foi de 72,40 segundos.

Ao fim do primeiro estudo de caso, foi utilizado o método *priority_objectives*, solicitando uma priorização entre os objetivos estratégicos, de modo a obter uma lista ordenada em critério de prioridade. Os resultados obtidos estão ilustrados a seguir:

Tabela 15 - Objetivos estratégicos ordenados em ordem de prioridade

Objetivo	**Descrição**
1	Estabelecer DisnAI como líder no entretenimento participativo global através de narrativas personalizadas;
2	Utilizar modelos generativos avançados para criar conteúdo dinâmico e adaptável ao feedback dos usuários, promovendo a criação colaborativa do enredo;
3	Garantir que o público seja um ator-criador fundamental na experiência imersiva da plataforma DisnAI;
4	Estender a presença global de DisnAI além dos urbanos, alcançando pessoas comuns em todo mundo e diversificando culturas e idiomas no conteúdo oferecido;
5	Formar parcerias estratégicas para expandir o alcance internacional da marca DisnAI através de empresas globais e influenciadores;
6	Investir continuamente

Fonte: Produzido pelo autor.

Embora tenha levado um tempo de processamento de 72,40 segundos, o modelo não alterou a ordem de objetivos, quando comparado à etapa anterior, também não efetuou qualquer alteração no texto. As hipóteses levantadas para este caso são: [1] O modelo, em etapa anterior, já considerou um ranqueamento com base no que ele entendia ser mais prioritário; ou [2] O modelo não consegue entender, com as informações fornecidas, o que defini a maior ou menor prioridade. Esta resposta, neste trabalho, é inconclusiva.

4.2 Estudo de Caso Comparativo - Mudanças no Parâmetro Performance

Alterando o parâmetro performance, variando entre performance 1 (200 *tokens*), performance 10 (2000 *tokens*) e performance 100 (20000 *tokens*), foi executado o mesmo *pipeline* de processamento feito na seção 4.1, com os mesmos inputs fornecidos, para avaliar as variações no desempenho do modelo em termos de consumo de tempo de processamento.

De maneira resumida, os tempos de processamento variando o número de *tokens* utilizados pelo modelo está ilustrado na tabela a seguir:

Tabela 16 - Diferenças no tempo de processamento para cada execução do modelo

Método Utilizado	**200 tokens**	**2000 tokens**	**20000 tokens**
fill_informations	67.21 s	100.41 s	106.45 s
response_quality_control [sem incluir respostas originais]	91.71 s	160.75 s	192.29 s
response_quality_control [incluindo respostas originais]	170.79 s	305.81 s	340.18 s
review_specific_response	21.85 s	27.39 s	42.61 s
review_target_with_comment	50.47 s	141.73 s	532.53 s
generate_strategic_objectives	191.79 s	503.06 s	4495.34 s
refine_strategic_objectives	65.69 s	212.24 s	138.19 s
priority_objectives	72.40 s	227.82 s	376.04 s

Fonte: Produzido pelo autor.

Observou-se que, em tempo de processamento, como esperado, a inicialização contando com 20000 *tokens* consumiu mais tempo que as demais, mas há ocorrências em que não há uma diferença tão significativa, e há ocorrências em que a diferença é altamente significativa. Por exemplo, observa-se que *fill_informations*, entre os 3 exemplos, não varia abruptamente, sendo a diferença entre a inicialização do modelo com 2000 tokens e com

20000 tokens sendo na ordem de 6 segundos apenas, o que pode, até mesmo, ser fruto da aleatoriedade (processos sendo executados em segundo plano no dispositivo local). Pode-se entender que, nestas circunstâncias, *fill_informations* não apresenta uma grande complexidade de resolução para o modelo, nem há tantas informações envolvidas, de modo que a mudança no número de *tokens* não impacta de forma tão acentuada no tempo de processamento.

Por outro lado, o método *generate_strategic_objectives* mostrou uma diferença significativa no seu tempo de execução, variando de 191,79 segundos (200 *tokens*) para 4495,34 segundos (20000 *tokens*), uma diferença relativa na ordem de 22,44 vezes. A interpretação dada a esta ocorrência diz respeito ao fato de que este método envolve um *prompt* e um desafio complexo, com muita informação envolvida, e um processo interpretativo que, mesmo para seres humanos, é complexo.

Analisando em termos do resultado final gerando pelos 3 modelos, são geradas as Tabelas 17 e 18, apresentando, respectivamente, os resultados dos objetivos estratégicos ranqueados pelos modelos inicializados com 2000 e 20000 tokens, respectivamente, sendo o resultado associado ao modelo com 200 tokens já apresentado na Tabela 15.

Tabela 17 - Objetivos estratégicos ordenados em ordem de prioridade [2000 *tokens*]

Objetivo	**Descrição**
1	Estabelecer DisnAI como líder inovador na indústria do entretenimento participativo ao longo dos próximos cinco anos;
2	Utilizar inteligentemente modelos generativos avançados para permitir a criação em tempo real com base nas interações dos usuários, diferenciando-se das outras plataformas atuais no mercado;
3	Construir uma comunidade digital forte e resiliente que promove empatia, inclusão cultural e diversidade por meio de histó0rias interativas ricas e imersivas;
4	Inspirar novas formas de expressão artística e narrativa digitalmente avançada que transcendam as fronteiras do entretenimento convencional;
5	Oferecer uma experiência única para os usuários residenciais e pessoas comuns, agregando valor significativo ao serviço consumido pelos mesmos;
6	Expandir continuamente a tecnologia generativa inteligente como ferramenta central da plataforma DisnAI;
7	Criar uma oportunidade exclusiva para os usuários participarem ativamente na construção coletiva das histórias interativas, sentindo-se como personagens-chave no processo;

8	Diferenciar a plataforma DisnAI por meio de sua capacidade única de permitir que cada indivíduo contribua para o desenvolvsemiento contínuo da narrativa em constante evolução;
9	Estabelecer uma base sólida com os usuários residenciais e pessoas comuns, fortalecendo a conexão entre eles através do engajamento colaborativo no processo criativo de histórias interativas;
10	Manter-se à frente do mercado por meio da inovação contínua na plataforma DisnAI e em suas ofertas para os usuários residenciais e pessoas comuns.

Fonte: Produzido pelo autor.

Tabela 18 - Objetivos estratégicos ordenados em ordem de prioridade [20000 *tokens*]

Objetivo	**Descrição**
1	Desenvolver novas ferramentas tecnológicas para facilitar a interação direta dos usuers com conteúdo;
2	Construir uma comunidade forte de fãs residenciais envolvidos ativamente na criação e evolução contínua da narrativa co-criada pela DisnAI, promovendo um senso de pertencimento e engajamento emocional;
3	Expandir a marca para atrair novas demografias além dos usuários residenciais por meio de parcerias estratégicas com influenciadores digitais que compartilham o conceito único da DisnAI, aumentando conscientização e alcance;
4	Investir em pesquisa e desenvolv0ment para manter a liderança no entretenimento participativo por meio de modelos generativos avançados, garantindo inovação contínua na criação do conteúdo co-criado pelos usuários;
5	Aumentar conscientização sobre o conceito único da DisnAI através de campanhas digitais focadas em participação e engajamento direto com potenciais clientes que valorizem a experiência colaborativa do entretenimento, além do consumo tradicional;
6	Estabelecer parcerias estratégicas com plataformas emergentes de mídia social para maximizar o alcance e promover novos usuários ao DisnAI que valorizem a participação direta na criação de conteúdo, expandindo assim as demografias atingíveis;
7	Garantir conformidade com regulamentações relacionadas à privacidade digital para proteger os dados dos usuários e manter confiança no uso da tecnologia generativa avançada pela DisnAI.;
Prioriti-zação:	The most important objective is to develop new technological tools that enable direct user interaction with content, as this forms the core of what makes DisnAI unique and essential for its business model. Building a strong community of active fans who are involved in co-creating narratives fosters

	engagement and emotional investment which can lead to organic growth and advocacy within their networks. Expanding brand reach through strategic partnerships with digital influencers aligns well with the startup's value proposition, as these individuals often have a significant impact on attracting new users who appreciate collaborative content creation experiences. Continuous investment in R&D ensures that DisnAI remains at the forefront of generative technology and maintains its competitive edge by consistently delivering innovative user-generated narratives, which is vital for long-term success. Increasing brand awareness through digital campaigns focused on participation directly targets potential clients who are likely to be attracted to DisnAI's unique offering of co-created entertainment experiences and can help in converting interest into active users. Establishing partnerships with emerging social media platforms is a strategic move for expanding the user base, as these demographics may not yet have been reached by traditional marketing efforts but are likely to be receptive due to their familiarity with digital collaboration and sharing culture. Lastly, ensuring compliance with privacy regulations builds trust among users who might otherwise hesitate about data security when engaging in online communities that handle personal information for content creation purposes.

Fonte: Produzido pelo autor.

Comparar a qualidade das respostas, em termos qualitativos não é um processo realizado neste trabalho, dada a natureza subjetiva deste processo. No entanto, percebe-se que, em número, a versão com 2000 *tokens*, gerou um maior número de objetivos quando comparado com as versões de 200 e de 20000 *tokens*. Isto não necessariamente significa um resultado melhor, haja vista que a versão de 20000 *tokens* pode ter investido mais esforços na ação de resumo e filtragem das informações.

Em geral, os modelos com 2000 e 20000 *tokens* não apresentaram o problema de respostas incompletas, como visto na versão de 200 *tokens*. Pode-se considerar, então, que a suspeita original, que as respostas incompletas estavam relacionadas ao número reduzido de *tokens*, se sustenta.

Alguns erros de escrita foram observados, como “histó0rias” ou “desenvolvsemiento”, que podem estar associados a um treinamento incompleto em língua portuguesa para estes modelos. Resultados assim eram esperados, considerando que modelos estão sendo majoritariamente treinados com conteúdos em língua inglesa, e carecem de modelos com altas capacidades de comunicação em língua portuguesa. O Phi3-Mini, por exemplo, é um dos poucos modelos de código aberto conhecidos no momento, que consegue dar respostas bem consistentes em língua portuguesa.

Um caso curioso, especificamente para o modelo inicializado com 20000 *tokens*, foi ele ter gerado um "objetivo estratégico adicional", uma espécie de narrativa, que ao invés de estar numerada e ranqueada (como os demais objetivos estratégicos) foi apresentada como "Prioritização" e escrito em língua inglesa, conforme visto ao final da Tabela 18. Em determinado aspecto, esta resposta pode ser entendida como um tipo de alucinação do modelo, dado que não estava sendo solicitada explicitamente esta narrativa final para o modelo, e está explicitamente solicitado que tudo deveria ser respondido em língua portuguesa. Neste aspecto, portanto, pode-se observar que aumentar o número de *tokens* por si só não garante um melhor resultado gerado pelo modelo, mas sendo notória a importância de verificações e controle de qualidade, estabelecidas nas rotinas desenvolvidas neste projeto.

5 – CONSIDERAÇÕES FINAIS

O presente trabalho explorou o potencial de Small Language Models (SLMs), com foco no modelo Phi3-Mini, para solucionar um problema complexo envolvendo processamento de linguagem natural, relacionado ao planejamento estratégico em startups. O trabalho consistiu em desenvolver uma aplicação orientada a objetos na linguagem Python, utilizando framework Ollama e modelo Phi3-Mini para gerar, refinar e priorizar objetivos estratégicos, tomando como base informações-chave de uma startup, como missão, visão, público-alvo, estágio do negócio, proposta de valor e vantagens competitivas.

Para isso, foram exploradas e utilizadas técnicas de *prompt engineering* e boas práticas vistas na literatura com relação ao desenvolvimento de modelos de LLM, dentre elas o controle de qualidade, a formatação e a interação via *prompts* para melhorar a qualidade de respostas fornecidas.

Mais precisamente com foco em SLMs, observou-se que estes modelos são potencialmente relevantes e conseguem se envolver e gerar respostas válidas em problemas complexos, mas, para isso, são necessários esforços em melhoria na qualidade dos *prompts* fornecidos. A estruturação do problema em etapas de processamento e todo o esforço em transformar conhecimento científico em um sequenciamento lógico, bem como a organização e a estrutura de *prompts* organizada por meio da classe desenvolvida, também se mostrou um resultado altamente significativo, que pode ser uma alternativa interessante a ser explorada em aplicações envolvendo SLMs no futuro: combinar conhecimento humano e lógica de programação com a capacidade de interpretação e generalização destes modelos, considerando *prompts* claros e processos estruturados em etapas.

A utilização da aplicação desenvolvida em ambiente local ocorreu sem qualquer travamento perceptível ou lentidão, de modo que o assistente teria condições de performar até mesmo como uma aplicação em segundo plano, e possivelmente até em configurações inferiores ao computador utilizado no trabalho.

É notável o destaque, o potencial e a evolução oriunda de LLMs, de modo que este desenvolvimento tecnológico deve ser incentivado. No entanto, com objetivo de diversificar a utilização destes modelos diretamente nos dispositivos, bem como com o objetivo de reduzir consumo energético e pegada de carbono para esta tendência tecnológica, são necessários esforços intensivos também na redução de consumo computacional e na otimização da eficiência destes modelos. SLMs, neste contexto, parecem uma alternativa interessante, podendo ser ainda mais exploradas no futuro.

Por meio da disponibilização pública do código-fonte da aplicação desenvolvida, este trabalho busca fomentar a pesquisa, o desenvolvimento e a inovação, incentivando a adoção de SLMs por empreendedores, pesquisadores e demais interessados. Essa iniciativa reforça a possibilidade de utilizar IA para resolver problemas estratégicos, ao mesmo tempo em que incentiva o uso de tecnologias acessíveis e replicáveis para propósitos de ciência, tecnologia e inovação.

REFERÊNCIAS

ABDIN, Marah et al. Phi-3 technical report: A highly capable language model locally on your phone. **arXiv preprint arXiv:2404.14219**, 2024.

BORTOLINI, Rafael Fazzi et al. Lean Startup: a comprehensive historical review. **Management decision**, v. 59, n. 8, p. 1765-1783, 2021.

CHIAVENATO, Idalberto. **Comportamiento organizacional**. 2009.

DA SILVA, Adriana Garcia; BUENO, Miriam Pinheiro; DE SOUZA, Rafael Rodrigues. Gerenciamento dos ativos de propriedade intelectual nas fases de estruturação das startups. **Revista de Gestão e Secretariado**, v. 14, n. 11, p. 20698-20709, 2023.

LIU, Pengfei et al. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. **ACM Computing Surveys**, v. 55, n. 9, p. 1-35, 2023.

LIU, Fei; KANG, Zejun; HAN, Xing. Optimizing RAG Techniques for Automotive Industry PDF Chatbots: A Case Study with Locally Deployed Ollama Models. **arXiv preprint arXiv:2408.05933**, 2024.

MEDEIROS, Gilvandro César de. **BUSINESS STRATEGY ENGINEERING (BSE): um método proposto para planejamento estratégico da inovação em startups**. 2023. Dissertação (Mestrado em Ciência, Tecnologia e Inovação) – Programa de Pós-Graduação em Ciência, Tecnologia e Inovação, Universidade Federal do Rio Grande do Norte, Natal, 2023.

MICROSOFT LATAM. *Pequeno, mas poderoso: pequenos modelos de linguagem Phi-3 têm grande potencial*. Disponível em: https://news.microsoft.com/source/latam/features/ia-pt-br/pequeno-mas-poderoso-pequenos-modelos-de-linguagem-phi-3-tem-grande-potencial/?lang=pt-br. Acesso em: 13 jan. 2025.

NAVEED, Humza et al. A comprehensive overview of large language models. **arXiv preprint arXiv:2307.06435**, 2023.

OGLI, Obloev Komronbek Hamza. PYTHON AND THE EVOLUTION OF PROGRAMMING PARADIGMS: A DEEP DIVE INTO VERSATILITY. **WORLD OF SCIENCE**, v. 7, n. 12, p. 49-55, 2024A.

OGLI, Obloev Komronbek Hamza. PYTHON AND ARTIFICIAL INTELLIGENCE: REVOLUTIONIZING DECISION-MAKING IN MODERN SYSTEMS. **WORLD OF SCIENCE**, v. 7, n. 12, p. 56-61, 2024B.

RIBAS, Ademir Juracy Fanfa; TEIXEIRA, Gylmar; FACINI, Marcio Alexandre. **Planejamento estratégico.** 2014.

RILLIG, Matthias C. et al. Risks and benefits of large language models for the environment. **Environmental Science & Technology**, v. 57, n. 9, p. 3464-3466, 2023.

SHANAHAN, Murray; MCDONELL, Kyle; REYNOLDS, Laria. Role play with large language models. **Nature**, v. 623, n. 7987, p. 493-498, 2023.

TOUVRON, Hugo et al. Llama: Open and efficient foundation language models. **arXiv preprint arXiv:2302.13971**, 2023.

WHITE, Jules et al. A prompt pattern catalog to enhance prompt engineering with chatgpt. **arXiv preprint arXiv:2302.11382**, 2023.

YIU, LM Daphne; YEUNG, Andy CL; JONG, Abe PL. Business intelligence systems and operational capability: an empirical analysis of high-tech sectors. **Industrial Management & Data Systems**, v. 120, n. 6, p. 1195-1215, 2020.

ZHAO, Wayne Xin et al. A survey of large language models. **arXiv preprint arXiv:2303.18223**, 2023.

ZHANG, Susan et al. Opt: Open pre-trained transformer language models. **arXiv preprint arXiv:2205.01068**, 2022.

ZHANG, Peiyuan et al. Tinyllama: An open-source small language model. **arXiv preprint arXiv:2401.02385**, 2024.

ZHU, Yichen et al. Llava-ϕ: Efficient multi-modal assistant with small language model. **arXiv preprint arXiv:2401.02330**, 2024.

ZIVOJINOVIC, Stevan. Balanced Scorecard As An Advanced Management Concept Within The Integrated Quality Management Model. **INTERNATIONAL JOURNAL FOR QUALITY RESEARCH**, v. 2, n. 1, p. 51-58, 2008.